

MSEI Publication Search

User Guide — OpenCode + NHR@FAU

Set up your computer to search the group's scientific-publication library from OpenCode.

This guide gets you searching the group's shared publication library from **OpenCode** on your own PC. You chat with an **NHR@FAU language model**, and a shared search server called **msei-publications** finds relevant passages, publications, and data across roughly **30,900 publications** and **351,000 full-text sections** — with real citations. Nothing is stored on your PC; the library lives on the group server and you reach it over the network.

What you need before you start

- **OpenCode** installed — <https://opencode.ai>
- Your **NHR@FAU API token** (from the NHR portal).
- The **server address** from your admin: `http://10.76.33.35:8080/mcp`

Installing OpenCode (it's a terminal app)

OpenCode runs in a **terminal** — there is no separate “software” vs “terminal” version. You install it once with a command, then start it by typing `opencode`. Configuring it (the token + config file) is the same however you launch it. Install with one of:

```
npm install -g opencode-ai      # anywhere Node.js is installed (easiest on Windows)
scoop install opencode          # or, Windows: Scoop
choco install opencode          # or, Windows: Chocolatey
curl -fsSL https://opencode.ai/install | bash  # macOS / Linux / WSL
```

Check it with `opencode --version`, then start it in your working folder with `opencode`. Inside OpenCode, `/mcp` lists connected servers — you'll see `msei-publications` after Step 2.

Step 1 — Set your NHR@FAU API token (Windows terminal)

Your model access is authenticated with an NHR **API token** — a secret key (technically a Bearer token, **not** an SSH login key). You store it once as a Windows environment variable so it never sits inside a config file.

1. Open **PowerShell** (Start → type “PowerShell” → Enter).
2. Set it for the current session — **the quotes are required**:

```
$env:NHR_API_TOKEN="PASTE_YOUR_TOKEN_HERE"
```

3. Verify it is set:

```
echo $env:NHR_API_TOKEN
```

4. Make it **permanent** for future terminals:

```
setx NHR_API_TOKEN "PASTE_YOUR_TOKEN_HERE"
```

5. Close and reopen PowerShell / OpenCode so the permanent value takes effect.

Important. Use the quotes — `$env:NHR_API_TOKEN="..."` works; without quotes it fails. `setx` only applies to **new** terminals, so restart OpenCode after it. **Never** paste your token into the config file, a screenshot, or a chat — if it is ever exposed, **regenerate it** in the NHR portal and set the new one.

(This is an API token for the model gateway. If you separately need an SSH key to log in to NHR HPC systems, see **Appendix B**.)

Step 2 — Configure OpenCode (models + search server)

OpenCode reads one JSON configuration file.

Where the file lives

- **Windows:** `%USERPROFILE%\config\opencode\opencode.json`
- **macOS / Linux:** `~/.config/opencode/opencode.json`

Create / open the config — Windows or macOS (terminal or by hand)

Windows — in PowerShell:

```
mkdir $env:USERPROFILE\config\opencode # ignore the error if it already exists
notepad $env:USERPROFILE\config\opencode\opencode.json
```

macOS / Linux — in Terminal:

```
mkdir -p ~/.config/opencode
open -e ~/.config/opencode/opencode.json # or: nano
~/.config/opencode/opencode.json
```

Prefer to do it **by hand**? Open that path in any text editor and create the file if it's missing (on macOS: Finder → Go → Go to Folder → type `~/.config/opencode`).

Paste this, then change X-User to your name and save

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "nhr-fau/Qwen/Qwen3.6-35B-A3B-FP8",
  "provider": {
    "nhr-fau": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "NHR@FAU",
      "options": {
        "baseUrl": "https://hub.nhr.fau.de/api/llmgw/v1",
        "apiKey": "{env:NHR_API_TOKEN}"
      },
    },
    "models": {
      "Qwen/Qwen3.6-35B-A3B-FP8": { "name": "Qwen3.6-35B" },
      "deepseek-ai/DeepSeek-V4-Flash": { "name": "DeepSeek-V4-Flash" },
      "RedHatAI/Mistral-Small-3.2-24B-Instruct-2506-FP8": { "name": "Mistral-Small-3.2-24B" },
      "google/gemma-4-E4B-it": { "name": "gemma-4-E4B-it" },
      "gpt-oss-120b": { "name": "gpt-oss-120b" }
    }
  }
},
```

```

    "mcp": {
      "mse-publications": {
        "type": "remote",
        "url": "http://10.76.33.35:8080/mcp",
        "enabled": true,
        "headers": { "X-User": "your.name" }
      }
    }
  }
}

```

Connecting to the VM — the details

Three things connect you to the server: the **URL** `http://<VM-IP>:8080/mcp` (the address from your admin — this deployment is `http://10.76.33.35:8080/mcp`; it must be `http` not `https`, and end in `/mcp`); `"type": "remote"` (a network server, not a local program); and `"X-User"` (your name — how you appear in the server log).

Already have an `opencode.json`? Add just this `mcp` block as a sibling of `provider` (don't create a second config file):

```

"mcp": {
  "mse-publications": {
    "type": "remote",
    "url": "http://10.76.33.35:8080/mcp",
    "enabled": true,
    "headers": { "X-User": "your.name" }
  }
}

```

What each part does

Field	Meaning
<code>model</code>	Default model, written <code>provider/model</code> . Use a strong tool-caller (Qwen3.6 or DeepSeek). Avoid gpt-oss-120b — it produces malformed tool calls.
<code>provider.nhr-fau</code>	The NHR@FAU model gateway.
<code>options.apiKey</code>	<code>{env:NHR_API_TOKEN}</code> reads the token from Step 1.
<code>models</code>	The exact model IDs your team may use, namespaced, e.g. <code>google/gemma-4-E4B-it</code> .
<code>mcp.mse-publications.url</code>	Search server address from your admin (ends in <code>/mcp</code>).
<code>headers.X-User</code>	How you appear in the server's usage log (use your name or initials).

It must be ONE object. A single `{` at the very top and `}` at the very bottom, with `model`, `provider`, and `mcp` as siblings (commas between them, no stray braces). An **EndOfFileExpected** error means a brace closed the file too early.

Step 3 — Activate the search tools

1. Save the config and **fully restart OpenCode**.
2. Open the **MCP panel** (the Servers / MCP view). You should see mseï-publications.
3. A **green dot** means connected. If it is **red**, see Troubleshooting. Toggle it **on** if needed.
4. Confirm it works — ask OpenCode:

```
Call corpus_stats on mseï-publications.
```

You should get counts (about **30,910 publications**, **351,010 chunks**). That means everything is connected.

If the model starts searching your local files instead of answering, it hasn't used the tool. Tell it explicitly: *"Use the mseï-publications MCP tools only; do not search the filesystem."* And make sure you are on Qwen3.6 or DeepSeek, not gpt-oss-120b.

Step 4 — Use it properly (with examples)

You ask in plain language; the assistant calls the right tool for you. The most useful tools:

Tool	What it does / when to use
search_text	Semantic search over full-text passages — the main search; finds specific statements with DOIs.
search_publications	Publication-level search over summaries — "find publications about X", with keyword/year filters.
evidence_pack	Gathers text (plus tables/figures) for one question at once — best for comprehensive answers.
get_publication	Everything about one publication by DOI — metadata, abstract, summary.
corpus_stats	What is in the library: totals, year range, top alloy systems and journals.
list_keywords	The controlled vocabulary you can filter by (alloy systems, techniques, etc.).
facet_counts	Distribution of a field, e.g. "top 15 alloy systems".
find_citing_publications	Citation graph — which corpus publications cite a given DOI.

Example prompts that work well

- *Using mseï-publications, search the full text for rhenium's effect on creep in nickel-base superalloys. Give the top passages with their DOIs.*
- *Find review publications on high-entropy alloys published after 2018 (filter kw_approach = review and year ≥ 2018).*
- *Build an evidence pack on gamma-prime rafting during creep, including any data tables.*

- *Get the full metadata and abstract for DOI 10.1016/j.actamat.2014.12.011.*
- *What are the top 15 alloy systems in the corpus?*
- *Which publications in the corpus cite 10.1016/j.actamat.2014.12.011?*

Tips

- For deep, passage-level answers, ask it to use `search_text` (full text) rather than `search_publications` (summaries).
- You can filter by keywords (alloy system, technique, year...). Ask “*what keywords can I filter by?*” → it calls `list_keywords`.
- Be specific. Vague prompts give vague searches.
- If a model misbehaves with tools, switch to **Qwen3.6** or **DeepSeek** in OpenCode’s model picker.
- Figures and tables are not loaded in this corpus yet, so those tools return nothing — text and summaries are fully searchable.

Troubleshooting

Symptom	Fix
EndOfFileExpected / JSON error	A brace closed the file early. It must be one object (model, provider, mcp as siblings). Validate at jsonlint.com .
401 Unauthorized / model errors	NHR_API_TOKEN not set, or OpenCode not restarted after <code>setx</code> .
“team not allowed to access model”	Use the exact namespaced model ID from the allowed list the error prints (e.g. <code>google/gemma-4-E4B-it</code>).
“Expected ‘id’ to be a string”	The model’s tool calls are malformed. Switch off <code>gpt-oss-120b</code> to Qwen3.6 or DeepSeek.
msei-publications red dot	Can’t reach the server. Open <code>http://10.76.33.35:8080/health</code> in a browser; if it fails, tell your admin.
It searches local folders instead of answering	The tool isn’t being used. Tell it “use the msei-publications MCP tools only”, and use a good tool-caller.

Quick reference

Set token (PowerShell):

```
$env:NHR_API_TOKEN="..."
setx NHR_API_TOKEN "..."
```

Config file:

```
%USERPROFILE%\config\opencode\opencode.json
```

Server health (open in a browser):

```
http://10.76.33.35:8080/health
```

First test, then a real search (ask OpenCode):

```
Call corpus_stats on msei-publications.  
Using msei-publications search_text, find <topic> and give DOIs.
```

Appendix A — For admins: you don't need to keep SSH open

The server runs as background Docker services on the VM and stays up **24/7**, independent of your SSH session, and restarts automatically after a reboot. SSH in only to administer it (update, check logs, load new data). Closing your SSH window does not stop the server.

Appendix B — (Optional) Generating an SSH key in Windows for NHR HPC

Only needed if you also require SSH access to NHR HPC systems (separate from the model gateway). In PowerShell:

```
ssh-keygen -t ed25519 -C "your.email@fau.de"
```

Press Enter for the default location (%USERPROFILE%\.ssh\id_ed25519). Your **public** key is id_ed25519.pub — upload that to the NHR portal. **Never share the private key** (id_ed25519, no .pub).